# AutoVaultLK

Second-Hand Vehicle Marketplace

Test Plan

Version 1.0

| | |
|---|---|
| Project ID(PID) | 11 |
| Group Number | 25 |
| Mentor | Mr. Bhanuka Siriwardhana |

**Group Members**

| **Registration No.** | **Name** |
|---|---|
| 230667F | Virusan T. |
| 230670H | Vishula J. |
| 230674A | R.P.M.Vithanage |

# Table of Contents


Group Members ..... 1
**1. Evaluation Mission and Test Motivation ..... 4**
1.1 Testing objectives ..... 4
**2. Target Test Items ..... 5**
2.1 Target Components to Test ..... 5
**3. Test Approach ..... 6**
3.1 Organization of test Approach ..... 6
3.2 Test Levels ..... 7
3.2.1 Unit Testing ..... 7
3.2.2 Integration Testing ..... 8
3.2.3 Contract Testing ..... 9
3.2.4 End-to-End Testing ..... 9
3.3 Testing Techniques and Types ..... 10
3.3.1 Data and Database Integrity Testing ..... 10
3.3.2 Function Testing ..... 12
3.3.3 User Interface Testing ..... 13
3.3.4 Performance Profiling ..... 14
3.3.5 Load Testing ..... 15
3.3.6 Security and Access Control Testing ..... 16
3.3.7 Failover and Recovery Testing ..... 17
3.3.8 Configuration Testing ..... 19
3.4 Level × Technique Coverage Matrix ..... 20
**4. Test Case Inventory ..... 21**
4.1 auth-user-service ..... 21
4.2 marketplace-service ..... 21
4.3 ingestion-service ..... 21
4.4 admin-service ..... 21
4.5 notification-service ..... 22
4.6 api-gateway ..... 22
4.7 web-frontend ..... 22
**5. Deliverables ..... 22**
5.1 Test Evaluation Summaries ..... 23
5.2 Reporting on Test Coverage ..... 24
**6. Risks, Dependencies, Assumptions, and Constraints ..... 25**
**7. References ..... 26**
**8. Responsibilities, Staffing and Training Needs ..... 27**
**8.1 Ownership by Technique ..... 27**
**8.2 Cross-Cutting Responsibilities ..... 28**
**8.3 Training Needs ..... 28**
**8.4 Staffing Assumptions and Constraints ..... 30**
**9. Test Environment Specification ..... 30**

**9.1 Environment Parity and Known Divergence....31**
**10. Entry, Exit and Suspension Criteria....31**
**10.1 Entry Criteria....31**
**10.2 Exit Criteria....32**
**10.3 Release Exit Criteria....34**
**10.4 Suspension Criteria and Resumption Requirements....34**
**11. Test Schedule and Phasing....35**
**11.1 Continuous Activities....36**
**12. Defect Severity and Priority Definitions....36**
**12.1 Severity Scale....36**
**12.2 Priority Scale....37**
**12.3 Required Fields and Lifecycle....37**
**13. Test Data Policy....38**
**13.1 Principles....38**
**13.2 Data by Environment....38**
**13.3 Adversarial and Negative Data....39**

# 1. Evaluation Mission and Test Motivation

AutoVaultLK is a cloud-native, services-based second-hand vehicle marketplace that connects buyers,dealers, and administrators around the buying and selling of used vehicles. The platform is built from a set of five independently deployable services, an authentication/user service, a marketplace service, an ingestion service, an admin service, and a notification service, fronted by a local API gateway for development, and AWS API Gateway in production ,and backed by PostgreSQL (with pgvector for semantic search), S3-style object storage, an SQS-style queue,and SES for email delivery.

The problem AutoVaultLK solves is the manual, error-prone, and slow process of listing large used-vehicle inventories one at a time. Dealers upload an entire inventory in one batch (CSV metadata plus a ZIP of images); the ingestion service parses, validates, normalises, and enriches that data, generates vector embeddings, and processes images, so that the marketplace service can expose the resulting catalogue to buyers through keyword, trigram, and vector-based semantic search.

Evaluation mission: verify the core marketplace flows end to end, find defects early in the authentication,search, ingestion, admin, and notification paths, and reduce release risk across all services and their integrations (S3, SQS, SES, JWT, and pgvector/search) before deployment.

Because the platform spans several independently deployable services, an asynchronous ingestion pipeline,external managed integrations, and machine-learning-derived search, defects can surface at many different layers a schema mismatch in a dealer's CSV, a malformed ZIP, a broken service-to-service or gateway contract, an authorization gap between user roles, configuration drift between environments, or a performance bottleneck during bulk ingestion. Testing at the unit, integration, contract, and end-to-end levels is therefore essential to catch these issues before they reach production.

## 1.1 Testing objectives

- Verify that all major functional requirements work correctly across authentication, marketplace, ingestion,admin, and notification services.
- Verify CSV inventory upload and processing, including validation of well-formed and malformed files.
- Verify ZIP image extraction and processing, including matching images to the correct vehicle records.
- Verify vehicle validation logic and correct persistence of vehicle data to the database.
- Verify embedding generation and the accuracy/availability of search - keyword, trigram, and pgvector-based semantic search, including ranking and fallback behaviour.
- Verify authentication and authorization across buyer, dealer, and administrator roles.
- Verify API Gateway routing, CORS, contract alignment, and security enforcement.
- Verify notification functionality triggered by system events (e.g. job completion, failures).
- Verify contract alignment between the API gateway, the web frontend, and backend services (public and internal OpenAPI specifications).
- Detects configuration drift across nginx, CORS, OpenAPI, and Terraform infrastructure definitions.
- Detecting defects before deployment, in line with finding as many important bugs as possible and assessing perceived quality risk.
- Verify database integrity, including constraints, relationships, and indexes.

- Verify error handling and recovery under failure conditions.
- Verify that the system delivers acceptable performance under normal and peak load.
- Track functional coverage against key requirements so testing completeness can be assessed and reported per area .

# 2. Target Test Items

The listing below reflects AutoVaultLK's actual service architecture. These are the software components, and the external integrations they depend on, that have been identified as targets for testing in this iteration.

## 2.1 Target Components to Test

| Target | Components to Test |
|---|---|
| auth-user-service | Registration, login, JWT issuance/validation, refresh and logout/session handling,password policy, role-based access, user and dealer profile operations |
| marketplace-service | Listing CRUD and lifecycle, favourites, search (filters, NL search, trigram/vector, ranking and fallback), image upload/serving, recommendations |
| ingestion-service | Upload initiation and job status, file parsing/chunking/validation/normalization/enrichment,embedding generation, image processing, queue/worker orchestration |
| admin-service | Admin reads and mutations, dashboard mapping, query DTO validation, internal service clients, protected administration endpoints |
| notification-service | Notification event handling, email template generation, SES adapter, retry sweeper logic |
| api-gateway | Public and internal OpenAPI contracts, CORS alignment, nginx routing/proxying, health checks, Terraform scaffold consistency |
| web-frontend | React UI, auth storage and RequireAuth behaviour, API contract tests, search/listing/dealer UI, recommendations, saved vehicles |
| Database / Schema | PostgreSQL schemas, tables, constraints, indexes, and pgvector-based search storage |
| External Integrations | S3 (object storage), SQS (queueing), SES (email), JWT (auth tokens), pgvector/search |

# 3. Test Approach

The test approach for AutoVaultLK is organised around the system's actual failure profile rather than a single uniform test style. The platform consists of five independently deployable services, a React single-page frontend, a schema-partitioned PostgreSQL database with vector search indexes, and Terraform-managed AWS infrastructure  and each of these fails in a different way. A schema-level defect (a bad migration, a missing grant) is invisible to a UI test; an authorisation gap is invisible to a database test; a Lambda cold-start or Step Functions wiring defect is invisible to both. The eight techniques in Section 3.1 are therefore each scoped to the layer at which their target class of defect is cheapest and most reliably detected, and together they cover data integrity, business-rule correctness, interface behaviour, performance under bulk ingestion and search load, security and access control, resilience to dependency failure, and configuration consistency across environments and browsers.

Both manual and automated implementations are used: automated suites (Jest/Vitest unit and integration tests, Supertest API journeys, k6 load scripts, and  once introduced  Playwright browser journeys ) carry the regression burden and gate every change in CI; manual technique is reserved for exploratory and usability assessment, dynamic security scanning triage, and the disaster-recovery and rollback rehearsals that are impractical to automate fully. A technique is judged useful and successful when it demonstrably locates defects at its intended layer, its oracle can be justified independently of the implementation under test, and it runs repeatedly without producing a misleading result when its required environment is unavailable.

The principal fault and failure models addressed are: functional/logical defects in business rules (incorrect pricing or eligibility logic); data-integrity failures (silent corruption, duplication or loss of vehicle records during ingestion); contract drift between the gateway, services and frontend; authentication and authorisation bypass; performance degradation under bulk-ingestion or search load; and failure to recover cleanly when a dependency such as PostgreSQL, S3, SQS, SES or the LLM provider becomes unavailable. Section 8 (Responsibilities, Staffing and Training Needs) is updated as each technique below is adopted, to record the environment and resourcing it requires.

## 3.1 Organization of test Approach

Test approach is organised along two independent axes, and the distinction between them is load-bearing rather than presentational. The first axis is the test level  where in the stack a test executes and what it is allowed to touch: unit, integration, contract, or end-to-end.

The second axis is the test technique or type which property of the system is being verified: data integrity, functional correctness, user-interface behaviour, performance, load tolerance, security, recoverability, or configuration consistency. Section 3.2 defines the levels; Section 3.3 defines the eight techniques; Section 3.4 shows where the two axes cross.

Separating the axes is necessary because in AutoVaultLK they genuinely do cross rather than nest. A single technique is routinely exercised at several levels, and a single level carries several techniques. Security and access control, for example, is verified at the unit level (a roles guard in isolation), at the integration level (per-service database privileges), at the contract level (CORS alignment and internal-endpoint exposure at the gateway), and at the end-to-end level (a dealer attempting to read another dealer’s upload job over HTTP). A plan that treated technique as a subdivision of level would have to describe that one technique four times over, and would obscure the fact that all four checks share a single oracle, the defined role permissions for buyer, dealer and administrator.

The system's architecture forces this separation. Because AutoVaultLK is a set of five independently deployable services rather than a monolith, an entire class of defect exists only in the space between services  a gateway route pointing at a path a service no longer exposes, a CORS origin permitted at the gateway but refused at the service, an internal endpoint reachable without internal credentials.

No unit suite can observe these, and reaching for an end-to-end test to catch them is both slow and imprecise. The contract level exists specifically to occupy that gap. Equally, because bulk inventory ingestion is asynchronous  a dealer's upload returns a job identifier immediately, and the CSV/ZIP is then parsed, validated, normalised, embedded and persisted by queue-driven workers  the correctness of an upload is simply not observable in the HTTP response that accepted it. Verifying it means asserting terminal job state, persisted rows, rejected record attribution and stage logs after the fact. Levels must therefore be defined by what each is able to observe, not merely by how much code each happens to cover.

Each level below states what it deliberately does not cover. Those exclusions are as much part of the plan as the inclusions: they are what prevents the levels from silently duplicating one another, what keeps the fast suites fast, and what makes it clear when a defect does escape to production which level should have caught it and must be strengthened.

## 3.2 Test Levels

Four levels are defined. Each is characterised below by its scope, the AutoVaultLK components it applies to, the tooling it uses, and its explicit exclusions.

### 3.2.1 Unit Testing

Unit testing verifies a single class, function, component or pipeline stage in isolation, with every process boundary collaborator database, HTTP client, S3, SQS, SES, LLM provider replaced by a test double. Suites mirror each service's source tree, so the test for any given source file is locatable by path. This is the level at which branch coverage, boundary values and error paths are pursued exhaustively, because it is the only level cheap enough to afford that thoroughness.

| Aspect | Specification |
|---|---|
| Applied to | ● All five NestJS services and the gateway: controllers, services, repositories, guards, strategies, mappers and DTO validators.<br>● In ingestion-service, each pipeline stage individually, file validation, row validation, parsing, chunk splitting, coercion and normalisation, enum vocabulary and dictionary snapshots, enrichment, embedding, image extraction and processing, persistence plus the stage graph and the concurrency controller that sequence them.<br>● In marketplace-service, the deterministic search parser (tokenisation, numerics, trigram matching, vocabulary, collision handling), ranking and sort-clause construction, and vector arithmetic.<br>● In web-frontend, React components, hooks, route guards and formatting utilities. |
| Tools | Jest with ts-jest and @nestjs/testing for the backend services and the gateway.<br>Vitest with jsdom, React Testing Library, @testing-library/user-event and @testing library/jest-dom for web-frontend.<br>Framework mocking facilities for test doubles; V8/Istanbul coverage reporters. |

| | |
|---|---|
| Deliberately does not cover | ● Real SQL execution. A query builder is asserted here on the SQL string it constructs, which cannot detect a cast PostgreSQL rejects or a column the owning service has since renamed; that is the integration level's responsibility.<br>● Composition of two or more units, and any wiring defect that appears only once modules are assembled.<br>● Cross-service behaviour, gateway routing and CORS alignment.<br>● Real browser rendering, layout, navigation history and file-upload dialogs. Timing, concurrency and throughput characteristics under sustained load. |

### 3.2.2 Integration Testing

Integration testing verifies that units compose correctly across module, process and schema boundaries, using real infrastructure wherever behaviour depends on it. Its defining characteristic in AutoVaultLK is a live, migrated PostgreSQL instance: suites assert on rows actually returned and persisted rather than on emitted SQL, and they connect as the owning service's own database role rather than as the database owner, so that a missing cross-schema GRANT fails a test rather than a deployment.

| Aspect | Specification |
|---|---|
| Applied to | ● Repository and query layers across all five schemas (auth, marketplace, ingestion, notification, admin), including cross-schema reads such as the marketplace listing-to-dealer join against auth.users and auth.dealer_profiles.<br>● The TypeORM migration set: forward application from an empty database, idempotency on re-run, and reversibility of the down-migrations.<br>● The ingestion pipeline composed across stage boundaries, and its persistence adapters for marketplace vehicles and vehicle images in particular upsert idempotency, so that re processing an identical dealer source file produces no duplicate listings.<br>● Search behaviour that only a real database exhibits: pgvector similarity casts and operators, trigram and text-search index selection, and ranking over genuinely stored embeddings. |
| Tools | Jest with the dedicated integration configuration (extended timeout, single worker) in marketplace-service and ingestion-service.<br>TypeORM DataSource and migration runner; the shared test-database bootstrap helper.<br>PostgreSQL with the pgvector and text-search extensions Docker Compose locally, a service container in CI.<br>Deterministic seed and fixture generators (vehicles, images, embeddings); psql and EXPLAIN ANALYZE for catalogue and query-plan inspection. |
| Deliberately does not cover | ● Business-rule branch coverage, already established at the unit level and not repeated here at substantially higher cost.<br>● Contract shape agreement between the gateway, the services and the frontend. Anything requiring a browser.<br>● Sustained concurrent load; these suites run single-worker precisely in order to remove concurrency as a variable. |

### 3.2.3 Contract Testing

Contract testing verifies that the published interfaces between independently deployable components remain mutually consistent. This level exists because AutoVaultLK's services are built and deployed separately: a change to one service can satisfy every one of that service's own tests and still break the gateway route or the frontend client that consumes it. Contract tests are deliberately cheap and fast so they can gate every change, and they assert agreement between declarations rather than exercising runtime behaviour.

| Aspect | Specification |
|---|---|
| Applied to | ● The api-gateway public and internal OpenAPI documents structural validity, and agreement between declared routes and the service paths they proxy to.<br>● CORS configuration alignment between the gateway and each backend service, since an origin permitted at one layer and refused at the other fails only in a real browser.<br>● The NGINX routing configuration, validated by the real NGINX parser rather than by inspection.<br>● web-frontend typed API client modules against the payload shapes the services actually emit (listings, ingestion, listing images, listing review, rejections).<br>● The ingestion/search normalisation and embedding parity check logic necessarily implemented on both sides of the pipeline, where silent divergence would degrade search quality with no other suite failing anywhere in the system.<br>● Emitted Lambda function configuration and the Step Functions state-machine definition, against the handlers and Terraform modules that consume them. |
| Tools | Jest, with @apidevtools/swagger-parser and js-yaml for OpenAPI validation.<br>The CORS synchronisation and NGINX validation scripts; containerised NGINX for configuration verification.<br>Stryker Mutator, scoped specifically to the shared/normalize-embed module, to guarantee the normalisation parity tests actually fail when implementations diverge.<br>The TypeScript compiler (tsc --noEmit) as a structural contract check across the frontend client and, for each service, across source and test code together necessary because the production build configuration excludes the test directories. |
| Deliberately does not cover | ● Whether the behaviour behind a correctly shaped contract is itself correct; a route may be perfectly declared and still return the wrong listings.<br>● Data persistence and database integrity. Authentication and authorisation enforcement at runtime.<br>● Performance of the contracted endpoints. |

### 3.2.4 End-to-End Testing

End-to-end testing verifies complete user journeys across the assembled system. Two sublevels are distinguished, because they observe different things. The API-level sublevel boots a service's real application module and drives journeys over HTTP through the full middleware, guard, validation, controller, service and repository stack. The browser-level sublevel, planned via Playwright test env drives the same journeys through a real browser and observes what only a browser exhibits: rendering and layout, navigation history, native file-upload dialogs, and session persistence across reloads.

| Aspect | Specification |
|---|---|
| Applied to | ● API level: per-service journey suites auth security and email-verification flows; marketplace listings, dealers, favourites, recommendations and search alias promotion; ingestion upload intake and job status; notification dispatch; admin protected endpoints.<br>● For the asynchronous ingestion path, journeys assert terminal job state, persisted vehicle and image rows, and rejected-record attribution after completion never the HTTP response that merely accepted the upload.<br>● Browser level (planned): the priority journeys buyer registration through email verification to first login; natural-language and filtered search through to vehicle detail and saving a favourite; dealer registration and profile completion; dealer bulk upload through real file selection, job-progress observation and rejections review; administrator login through dashboard, user management and audit-log review; and session expiry with re-authentication. |
| Tools | API level: Jest with each service's jest-e2e configuration, Supertest, and the @nestjs/testing application factory against a seeded test database, with LocalStack (or Testcontainers) to emulate real AWS S3, SQS, and SES semantics locally and in CI.<br>Browser level: Playwright for end-to-end user journeys; leveraging its built-in auto-waiting to eliminate flaky sleeps, network interception to securely stub Groq/LLM calls for deterministic NL-search testing, and execution across Chromium, Firefox, and WebKit (Safari) |
| Deliberately does not cover | ● cover Logic, validation rules and boundary conditions already proven at the unit and integration levels; re-verifying them here is slow, and a failure at this level is far harder to localise.<br>● Exhaustive permutation of any kind this level covers journeys, not combinations. Sustained concurrent load, which the Load Testing technique addresses against staging.<br>● Infrastructure and configuration drift, which the contract level addresses more directly and far earlier. |

## 3.3 Testing Techniques and Types

### 3.3.1 Data and Database Integrity Testing

AutoVaultLK's commercial value is its inventory, and that inventory reaches the database almost entirely through an automated pipeline rather than through human data entry. A dealer uploads a CSV and a ZIP once; everything thereafter parsing, normalisation, enrichment, embedding, image association, persistence happens without anyone looking at the rows. A defect that duplicates a listing on re-upload, drops a record silently, or writes a vehicle against the wrong dealer is therefore invisible at every other layer: the upload still returns success, the job still reports completion, and the catalogue still renders. This technique exists because the database is the only place those failures are observable, and because five services share one instance across five schemas, where a missing cross-schema grant is a production outage waiting for its first request.

| | |
|---|---|
| **Technique Objective:** | Exercise AutoVaultLK's database layer schema, migrations, constraints, indexes and repository queries across the auth, marketplace, ingestion, notification and admin schemas independently of the UI, so that data corruption, incorrect persistence, or unauthorized cross-schema access is observed and logged directly rather than inferred from application behaviour. |
| **Technique:** | ● Run the full TypeORM migration set against an empty PostgreSQL instance and assert the resulting catalogue (tables, columns, types, constraints, foreign keys, enum vocabularies, and indexes, including the pgvector similarity index and the trigram/text-search indexes); re-run against an already-migrated database to confirm idempotency, then revert and re-apply to confirm the down-migrations are honest.<br>● Execute each service's repository queries (listings, favourites, search, upload jobs, rejected records, notifications, audit logs) against a seeded, migrated database, connecting as the owning service's own database role never as the database owner so that a missing GRANT across schema boundaries is caught by the test rather than by a deployment.<br>● Exercise the ingestion pipeline's persistence adapters end-to-end: re-process an identical CSV/ZIP source to confirm upsert idempotency (no duplicate vehicle listings), verify rejected-record capture with stage attribution, verify image-to-vehicle association, and verify provenance is recorded for LLM-normalised fields.<br>● Seed deliberately invalid data duplicate VINs, out-of-range year/mileage/price values, malformed enum values, orphaned foreign keys to confirm each constraint and unique/partial-unique index rejects what it is meant to reject. |
| **Oracles:** | ● Direct catalogue inspection (via psql / information_schema) checked against the migration set as the declared source of truth for schema state<br>● Row-level assertions on data actually returned or persisted never assertions on the SQL string emitted.<br>● Invariant checks that must hold regardless of path: no orphan rows, no duplicate listings after re-ingestion, every rejection attributed to a pipeline stage, every normalised field carrying provenance.<br>● EXPLAIN ANALYZE output confirming that search and listing queries use their intended index rather than a sequential scan. |
| **Required Tools:** | ● Jest with a dedicated integration configuration, and TypeORM's DataSource and migration runner<br>● PostgreSQL with the pgvector and text-search extensions (Docker Compose locally; a service container in CI)<br>● psql, used directly for catalogue and EXPLAIN ANALYZE inspection<br>● Deterministic seed and fixture generators, plus a role-verification script that connects as each service's own database role |
| **Success Criteria:** | Every schema (auth, marketplace, ingestion, notification, admin) has its migrations proven forward, idempotent and reversible; every cross-schema read and upsert path used by a service is covered by a live-database test; a missing or incorrect grant fails a test rather than a production deployment; re-processing an identical dealer source file produces zero duplicate vehicle records. |

| | |
|---|---|
| **Special Considerations:** | ● Integration suites run single-worker and skip rather than fail when no local PostgreSQL instance is reachable, so a missing local dependency never produces a misleading red build.<br>● Suites own and clean up their own fixtures so they remain order-independent and repeatable.<br>● Constraint-negative seeding is performed through the ORM/SQL layer only, never through the application UI, so the database itself not the frontend's validation is what is being proven. |

### 3.3.2 Function Testing

Dealer verification gates listing creation; upload validation must reject a malformed CSV row without abandoning the remainder of the batch; and search must resolve a free-text query through tokenisation, vocabulary lookup, trigram matching and vector ranking, with a deterministic fallback when the LLM path is unavailable. This technique is the primary evidence that each of those rules behaves as specified for both valid and invalid input, and it is deliberately spread across the unit and end-to-end levels because the rules themselves are cheapest to prove in isolation while their composition into a working journey is only observable over HTTP.

| | |
|---|---|
| **Technique Objective:** | Exercise the functional behaviour of each AutoVaultLK service auth-user, marketplace, ingestion, admin, notification and api-gateway against the SRS use cases, verifying correct processing and persistence for valid input and correct error/warning behaviour for invalid input. |
| **Technique:** | ● Unit suites (Jest/ts-jest, @nestjs/testing) mirror each service's source tree, replacing the database, S3, SQS, SES and LLM collaborators with test doubles; cases are derived by equivalence partitioning and boundary-value analysis, e.g. CSV row-validation limits, price/year/mileage bounds, and enum/vocabulary edges.<br>● API-level end-to-end suites (Supertest) boot each service's real application module and drive full journeys over HTTP dealer registration through verification, login and bulk upload to job-status; buyer search through to saving a favourite; administrator review through to an audit-log entry asserting response codes/bodies and the resulting database state.<br>● Business-rule cases are executed with both valid and deliberately invalid data (malformed CSV rows, mismatched or oversized ZIP contents, out-of-range price/mileage) to confirm the correct error or warning is returned and no partial record is left behind.<br>● Natural-language search parsing, trigram matching and vector ranking are each tested against independently derived expected output not derived from the implementation including the deterministic-parser fallback path exercised with the LLM path forced unavailable. |
| **Oracles:** | ● The SRS as the specification of expected behaviour.<br>● The public and internal OpenAPI contracts for expected request/response shape and status codes.<br>● Resulting database state, queried after the call, as the authority for side effects (a vehicle actually persisted, a notification actually queued).<br>● Independently computed expected values for parsing, ranking and normalisation logic, rather than values read back from the code under test. |
| **Required Tools:** | ● Jest with ts-jest and @nestjs/testing (unit and module-level tests) |

| | |
|---|---|
| | ● Supertest for API-level journey testing<br>● A seeded test database per service<br>● Test doubles / local emulators for S3, SQS, SES and the LLM provider |
| **Success Criteria:** | Every functional requirement is exercised by at least one passing test; every use-case flow has both a valid-data case and an invalid-data case; every business rule (pricing bounds, dealer-verification gating, upload validation) has an explicit assertion. |
| **Special Considerations:** | ● Journeys depending on email verification or job-status polling use fixture or short-circuit mechanisms rather than waiting on real delivery delay.<br>● LLM-dependent paths are additionally tested with the fallback path forced, since live model output is not a stable oracle. |

### 3.3.3 User Interface Testing

AutoVaultLK presents three quite different interfaces from one React application: a public buyer surface that must be usable by anyone arriving from a search engine, a dealer console whose bulk-upload and rejections-review screens are the primary workflow for the platform's paying users, and an administrative surface whose actions are audited. Each carries state the others do not: a dealer awaiting verification, an upload job mid-progress, an empty saved-vehicles list and those intermediate states are where interface defects concentrate. This technique verifies each surface across its full state space rather than on the populated happy path alone, and asserts through the accessibility tree so that what is verified is what a user can actually perceive and operate.

| | |
|---|---|
| **Technique Objective:** | Exercise navigation, forms and interactive objects across the web-frontend's buyer, dealer and administrator surfaces to observe and log standards conformance accessible roles/labels, keyboard operability, focus order and correct target behaviour of each window/component and its states. |
| **Technique:** | ● Component tests (Vitest with jsdom and React Testing Library) render each page, component and hook across its state space initial, loading, populated, empty, error, unauthorised locating elements by accessible role, label and text, and driving them through @testing-library/user-event sequences (tab order, keyboard activation, form entry) rather than through internal state.<br>● Route-guard tests verify navigation and redirect behaviour for RequireAuth and dealer-type/verification-state gating across authenticated, unauthenticated and wrong-role states.<br>● A planned Playwright suite, structured on the Page Object Model, drives the priority journeys registration through verification and first login; search through vehicle detail to saving a favourite; dealer bulk upload through job-progress observation to rejections review; administrator login through dashboard and audit-log review in real browsers, observing rendering, real file-upload dialogs and session persistence that a simulated DOM cannot. |
| **Oracles:** | ● The design/wireframe documentation for intended layout and interaction.<br>● The accessibility tree: required roles, labels, names and focus order must be present and correctly ordered.<br>● Explicit assertions on rendered text/element presence and absence, disabled and invalid states, and navigation outcome. |

| | |
|---|---|
| | ● Recorded API-client calls as the oracle for interaction side effects, with network intercepted at the client boundary so results are deterministic. |
| **Required Tools:** | ● Vitest, jsdom, React Testing Library, @testing-library/user-event, @testing-library/jest-dom<br>● React Router test utilities for guard/redirect behaviour<br>● Playwright, executing locally and in CI across Chromium, Firefox, and WebKit engines. (planned) for the browser-based layer |
| **Success Criteria:** | Every page, form and guard has a suite covering loading, empty, error and populated states; every form has one valid-submission case and one case per validation failur |
| **Special Considerations:** | ● Component suites intercept network calls so they remain deterministic and offline-capable; not every custom or third-party rendering detail (e.g. exact pixel layout) is asserted at this layer. |

### 3.3.4 Performance Profiling

Two operations in AutoVaultLK have cost profiles that cannot be reasoned about from the code: semantic search, whose latency depends on pgvector index selection and grows with corpus size, and bulk ingestion, whose per-record cost is dominated by LLM normalisation calls, embedding generation and Sharp image processing, all executed under AWS Lambda with cold starts and a fixed memory ceiling. Neither is measurable on a developer laptop with a hundred seeded vehicles. This technique establishes quantified single-user baselines for each critical transaction before any concurrent load is applied, so that the load results in the following section can be attributed to a specific component rather than merely observed.

| | |
|---|---|
| **Technique Objective:** | Exercise AutoVaultLK's critical transactions structured and natural-language search, listing retrieval, favourites, dealer bulk ingestion and notification delivery under normal anticipated workload and single-user conditions to establish quantified response-time and resource-usage baselines against the non-functional requirements. |
| **Technique:** | ● Before any load is applied, profile the components whose cost dominates individually: database query plans (EXPLAIN ANALYZE) for every search and listing query; pgvector similarity-search cost as corpus size grows; LLM-assisted normalisation/parsing latency including the deterministic fallback; embedding-generation throughput; Sharp image-processing cost per image; and AWS Lambda cold-start duration and memory headroom for the ingestion pipeline.<br>● Run single-user, single-transaction scripts for each key operation to establish a best-case per-transaction baseline (built from the same scripts used for Function Testing, with unnecessary interaction delay removed), then repeat with multiple concurrent virtual clients at expected normal load.<br>● Measure p50/p95/p99 response-time percentiles and compare them against the stated non-functional requirements, not against whatever the system currently achieves. |
| **Oracles:** | ● The non-functional requirements as the authority for target percentile response times and throughput.<br>● Recorded baselines as the authority for future regression, with a defined |

| | |
|---|---|
| | tolerance band.<br>● EXPLAIN ANALYZE output confirming index usage rather than sequential scan for every profiled query. |
| **Required Tools:** | ● k6 (scriptable, CI-friendly, threshold-based pass/fail gates)<br>● EXPLAIN ANALYZE and pg_stat_statements for database profiling<br>● AWS CloudWatch metrics, Lambda Insights and X-Ray for cold-start and distributed-latency attribution<br>● Lighthouse for frontend Core Web Vitals on buyer-facing pages |
| **Success Criteria:** | A documented baseline exists for structured search, natural-language search, listing detail, favourites, dealer bulk ingestion and notification delivery; single-user and expected-concurrency runs both complete without any failure attributable to the test setup itself. |
| **Special Considerations:** | ● Profiling and baseline runs are performed against the staging (AWS) environment at representative data volume, since instance sizing, network topology and managed-service limits are not reproducible on a developer laptop.<br>● A background workload is deliberately not simulated at this stage sustained concurrent load is the concern of Load Testing (section 3.3.5). |

### 3.3.5 Load Testing

The load profile of this platform is asymmetric in a way that shapes the whole technique. Buyer search is high-frequency, low-cost and latency-sensitive; dealer bulk ingestion is infrequent, extremely expensive per invocation, and tolerant of seconds or minutes but the two share a database, a Lambda concurrency budget and a connection pool. The risk is therefore not that either fails alone, but that one dealer uploading a large inventory degrades search for every concurrent buyer. This technique deliberately includes volume testing of progressively larger CSV/ZIP batches alongside conventional load, stress, spike and soak profiles, in order to locate that interaction and document the practical per-job ceiling.

| | |
|---|---|
| **Technique Objective:** | Subject AutoVaultLK's key transactions buyer search, listing retrieval, and especially dealer bulk-inventory ingestion to varying workload conditions, from expected average load up through and beyond anticipated peak, to determine that the system continues to function correctly and to evaluate its performance characteristics under each condition. |
| **Technique:** | ● Load expected concurrency sustained over a defined period, asserting that percentile latency thresholds hold throughout.<br>● Stress concurrency ramped past expected peak to locate the saturation point and confirm the system degrades via queueing, throttling or clear errors rather than opaque failure.<br>● Spike abrupt bursts of arrival traffic, representative of a marketing push, to validate autoscaling and Lambda cold-start behaviour under sudden load.<br>● Soak extended moderate load to expose memory growth, connection-pool exhaustion or unbounded caches.<br>● Volume ingestion of progressively larger CSV/ZIP source files and record counts to establish the practical per-job ceiling and confirm |

| | |
|---|---|
| | chunking and concurrency control hold at scale.<br>● Workloads are built from the transaction scripts developed for Function Testing (with unnecessary interaction delay removed) and executed against the staging environment, sized to mirror the production topology. |
| **Oracles:** | ● Percentile latency and error-rate thresholds asserted by the load generator itself, so a run passes or fails without manual interpretation.<br>● Resource metrics CPU, memory, database connection-pool utilisation, SQS queue depth, Lambda concurrency checked against configured limits.<br>● A documented saturation point beyond which degradation is expected and predictable, rather than treated as an anomaly. |
| **Required Tools:** | ● k6 as the primary load generator, with thresholds configured as pass/fail gates<br>● AWS CloudWatch dashboards and alarms for resource observation during runs |
| **Success Criteria:** | All percentile targets are met under expected load; the saturation point is known and documented, with degradation past it predictable rather than opaque; no memory growth or connection-pool leak is observed over the soak period; the maximum practical ingestion batch size is documented. |
| **Special Considerations:** | ● Load testing is run on a dedicated staging environment at a scheduled time, to permit full control and accurate measurement, and is never run against production.<br>● Test data volume is scaled to match anticipated production inventory size, so results are representative rather than optimistic. |

### 3.3.6 Security and Access Control Testing

AutoVaultLK holds commercially sensitive data under three distinct roles whose boundaries are not merely cosmetic: a dealer's inventory, pricing and upload history must be invisible to every other dealer, while administrators can read across all of them. On top of that, the services trust one another over internal endpoints, and the ingestion pipeline accepts dealer-supplied archives and passes dealer-supplied text to an LLM. The attack surface is consequently wider than a conventional CRUD application's: it includes object-level authorisation between peers of the same role, internal endpoints that must reject uncredentialed callers, archive extraction that must resist path traversal, and prompt injection through normalisation. This technique addresses all of them explicitly.

| | |
|---|---|
| **Technique Objective:** | Exercise AutoVaultLK's authentication, role-based access control, internal service-to-service trust boundaries and transport configuration to confirm that a buyer, dealer or administrator can access only the functions and data their role permits, and that only credentialed callers can reach internal endpoints or the system at large. |
| **Technique:** | ● Application-level security: for every protected endpoint, list each role (buyer, dealer, administrator, unauthenticated) and its intended permissions; create an explicit permitted case and an explicit denied case for each, then re-run the same transaction as a different role/user to confirm access is correctly granted or denied. Object-level authorisation is verified directly by identifier substitution e.g. one dealer attempting to read or mutate another dealer's listings or upload jobs.<br>● System-level security: verify JWT issuance, signature, expiry and claim |

| | |
|---|---|
| | validation; verify rejection of absent, malformed, expired, wrongly-signed and algorithm-substituted tokens; verify refresh-token rotation including reuse detection and family invalidation on replay; verify that internal service-to-service endpoints reject calls lacking valid internal credentials.<br>● Input-handling checks: DTO validation fuzzed with over-long, wrongly-typed, missing and malformed fields and with SQL/script metacharacters; uploaded CSV/ZIP files tested for type, size and content-mismatch validation and for path-traversal/archive-abuse resistance during extraction.<br>● Abuse protection: rate-limit and lockout behaviour on login, registration, verification-resend and password-reset endpoints, including correct counter reset after the lockout window.<br>● Transport hardening: security-header presence, cookie attributes (HttpOnly, Secure, SameSite), and CORS origin restriction checked at both the API gateway and each individual service. |
| **Oracles:** | ● The security requirements in the SRS/SAD, and the OWASP Top 10 as an external authority for expected control behaviour.<br>● Explicit assertion that unauthorised access yields the correct rejection status code with no data disclosure in the response body.<br>● Response and log content asserted free of credentials, tokens and password hashes. |
| **Required Tools:** | ● Jest and Supertest for authentication, authorisation and abuse-protection suites<br>● OWASP ZAP for baseline and authenticated active scanning against staging (planned)<br>● npm audit and automated dependency-vulnerability alerts<br>● The role-verification script for confirming database-level privileges per service role |
| **Success Criteria:** | Every protected endpoint is covered for every role (buyer, dealer, administrator, unauthenticated), including denial cases; object-level authorisation is proven by identifier-substitution tests on every owned resource (listings, upload jobs, dealer profiles); token replay/rotation/expiry behaviour is proven; internal endpoints reject uncredentialed calls; no unremediated high or critical scanner finding remains before release. |
| **Special Considerations:** | ● Active/authenticated scanning against a deployed environment is performed only against staging, never against production.<br>● Probing/fuzzing tools are run by the QA function under controlled conditions and scheduled so they do not interfere with concurrent load-testing exercises. |

## 3.3.7 Failover and Recovery Testing

The ingestion pipeline depends on five external services PostgreSQL, S3, SQS, SES and a third-party LLM provider any of which can time out, throttle or fail mid-batch, and it runs under Lambda where a timeout can terminate a worker part-way through a chunk. The unacceptable outcome is not a failed job, which is recoverable, but a job that reports success over a partially written batch, which is not: the dealer believes their inventory is live and no alarm fires. This technique therefore concentrates on proving that every failure path leaves the database in a defined state, that replaying any upload or queue message has exactly-once effect, and that natural-language search degrades to its deterministic parser rather than failing outright when the LLM path is unavailable.

| | |
|---|---|
| **Technique Objective:** | Simulate failure of AutoVaultLK's dependencies PostgreSQL, S3, SQS, SES, the LLM provider, and peer services and exercise the recovery processes, so that the system is shown to degrade predictably, recover completely once the dependency returns, and never report success for a partially processed ingestion batch. |
| **Technique:** | ● Dependency-failure injection: deliberately fail each external dependency database connection exhaustion, S3 read/write rejection, SQS unavailability, SES delivery rejection, LLM timeout/rate-limit/error, peer-service 5xx or timeout via failing test doubles at the unit layer, and via Docker Compose container start/stop/pause at the environment layer.<br>● Graceful-degradation testing: confirm natural-language search falls back to the deterministic parser when the LLM path is unavailable, and that the fallback ranking remains useful rather than failing the page.<br>● Retry and idempotency testing: replay the same upload, queue message, or notification event and assert exactly-once effect; confirm bounded retry with backoff and correct dead-letter routing for exhausted messages, with the original payload preserved for reprocessing.<br>● Pipeline-interruption testing: interrupt an ingestion job at each stage boundary including a simulated Lambda timeout and a Step Functions state transition and confirm the job resumes or fails cleanly with accurate stage logs and job status.<br>● SQS/S3 visibility timeouts, redeliveries, and edge cases are simulated locally using LocalStack fault injection<br>● Infrastructure recovery: rehearse service restart, database connection-pool recovery after a PostgreSQL restart, and backup/point-in-time restoration against a non-production environment, measuring recovery time and data loss against the stated RPO/RTO; rehearse a migration-bearing deployment rollback. |
| **Oracles:** | ● The stated RPO and RTO as the authority for acceptable data loss and downtime.<br>● Post-failure database state asserted to contain no partial write and no lost committed record.<br>● Job and stage-log state asserted to reflect reality never reporting success for a partially processed batch.<br>● Dead-letter contents are asserted to preserve the original payload for reprocessing, and CloudWatch alarm-state transitions are asserted to occur when the corresponding fault is injected. |
| **Required Tools:** | ● Jest with failure-injecting test doubles and dedicated error-path suites<br>● Docker Compose for controlled dependency start, stop and pause<br>● AWS Step Functions execution history and CloudWatch Logs for pipeline-state evidence<br>● Database snapshot, restore and point-in-time recovery facilities; CloudWatch alarms and dashboards |
| **Success Criteria:** | Every external dependency has a failure test asserting a clear error, a diagnostic log entry, and no partial write; replay of any message, upload or notification produces exactly-once effect; an ingestion job interrupted at any stage boundary always resumes or fails cleanly; a database restore meets the stated RPO/RTO in a timed rehearsal; a migration-bearing release is proven rollback-capable; every critical alarm is proven to fire under its triggering fault. |
| **Special** | ● Recovery testing is intrusive by nature; dependency-outage simulation is |

| | |
|---|---|
| **Considerations:** | performed against Dockerised local services or the staging environment, never against production.<br>● Exercises are scheduled outside of concurrent load-testing windows, and resources from database and infrastructure ownership are required to corrupt state and rehearse restoration safely. |

### 3.3.8 Configuration Testing

AutoVaultLK's routing and access rules are declared in four places that must agree but have no shared source of truth: the NGINX configuration, the public and internal OpenAPI documents, each service's own CORS configuration, and the Terraform definitions for three environments. Drift between any two of them produces a defect that is invisible locally and appears only after deployment a route proxying to a path that no longer exists, an origin the gateway permits and the service refuses. This technique exists to catch that drift before deployment rather than after, and it additionally covers the supported browser matrix, since the buyer surface is the one part of the system whose behaviour genuinely differs between rendering engines.

| | |
|---|---|
| **Technique Objective:** | Exercise AutoVaultLK across its supported deployment and client configurations the dev, staging and production Terraform environments, the Node.js runtime version, and the Chromium, Firefox, and WebKit matrix to observe target behaviour under each configuration and to detect drift between the configuration layers (gateway, service, and infrastructure) that must otherwise agree. |
| **Technique:** | ● Validate and plan each Terraform-defined environment (dev, staging, production) independently, confirming terraform validate/plan shows no unreviewed destructive change and that each environment's variable set (database endpoints, S3 buckets, SES sender identity, LLM provider keys) resolves correctly.<br>● Run the priority browser journeys from section 3.3.3 against each entry in the supported browser matrix (Chrome and Firefox as primary, Edge as secondary; desktop and mobile-emulated viewports) to detect rendering or behavioural differences between engines.<br>● Cross-check the API gateway's CORS configuration, its public/internal OpenAPI route definitions, and the NGINX routing configuration against the definitions actually consumed by each backend service, so drift between any two of the three is caught before deployment.<br>● Execute mutation testing (Stryker) strictly on the ingestion/search normalisation parity check to ensure silent drift is caught.<br>● Verify Lambda packaging and emitted function configuration against the Terraform Lambda module definitions that reference them, and verify the Step Functions state-machine definition matches its handlers. |
| **Oracles:** | ● Terraform's own validate/plan output, with destructive changes treated as findings requiring justification.<br>● The architecture/deployment documentation as the authority for intended topology.<br>● The OpenAPI documents' and the NGINX configuration's own parser verdict as the authority for routing correctness.<br>● Observed rendering and behavioural differences across the supported browser matrix. |

| | |
|---|---|
| **Required Tools:** | ● Terraform fmt, validate and plan<br>● Docker, for containerised NGINX configuration verification<br>● Jest for scaffold-consistency and Lambda-function-configuration suites<br>● Playwright for cross-browser verification. (planned) for cross-browser verification; AWS CLI for post-deployment smoke checks per environment |
| **Success Criteria:** | Every Terraform environment plans without unreviewed destructive change; CORS origins and OpenAPI route definitions match across the gateway and every service; the NGINX configuration passes its own validator; the priority journeys render and behave consistently across the supported browser matrix; Lambda bundle configuration matches the Terraform definitions that consume it. |
| **Special Considerations:** | ● Configuration testing depends on access to all three Terraform-managed environments and cannot be fully exercised from a developer's local machine alone. |

## 3.4 Level × Technique Coverage Matrix

The matrix below shows where each of the eight techniques defined in Section 3.3 is exercised across the four levels defined in Section 3.2. It is included to make the crossing of the two axes explicit: reading across a row shows that a single technique is pursued at several levels with different oracles and different cost, and reading down a column shows that a single level carries several techniques at once. P marks the level at which a technique is primarily established, and where its defects are cheapest to find; S marks a level where the technique is exercised in a secondary or confirmatory role; – marks a combination deliberately not pursued, consistent with the exclusions stated in Section 3.2.

| Technique (section 3.3) | Unit | Integration | Contract | End-to-End |
|---|---|---|---|---|
| 3.3.1 Data and Database Integrity | S | P | – | S |
| 3.3.2 Function | P | S | – | P |
| 3.3.3 User Interface | P | – | S | P |
| 3.3.4 Performance Profiling | S | S | – | P |
| 3.3.5 Load | – | – | – | P |
| 3.3.6 Security and Access Control | P | S | S | P |
| 3.3.7 Failover and Recovery | S | S | – | P |
| 3.3.8 Configuration | – | S | P | S |

Three features of the matrix drive the rest of this plan. First, no technique is confined to a single level except one, which is the substantive justification for separating the two axes rather than nesting them. Second, that exception is Load Testing: it is meaningless below the assembled system, and it is therefore run against staging at representative data volume rather than in CI. Third, Data and Database Integrity is primary at the integration level rather than the unit level, because a unit test of a query builder asserts the SQL string and so cannot observe the failures that matter most here a pgvector cast

the database rejects, a similarity function with transposed arguments, or a cross-schema join to a renamed column.

# 4. Test Case Inventory

This section lists the concrete test case areas identified per service, based on a review of the current codebase. These items feed directly into the unit, integration, contract, and end-to-end test levels described in Section 3, and form the basis of the functional coverage reporting described in Section 5 .

## 4.1 auth-user-service

- Registration, login, JWT validation, refresh flow, logout/session handling
- Password policy, invalid credentials, expiry/rotation, role-based access
- User profile and dealer profile operations
- Security-focused checks such as guards, strategies, and auth e2e/security flows

## 4.2 marketplace-service

- Listing CRUD and listing lifecycle
- Dealer profile read paths
- Favourites add/remove/list
- Search filters, search stats/options, vehicle detail search, alias promotion
- Natural-language search, deterministic parser, trigram/vector search, ranking/fallback behavior
- Image upload, safe path handling, and image serving
- Recommendations generation and retrieval
- JWT auth, roles guard, current-user decorator, and auth-related unit tests

## 4.3 ingestion-service

- Upload initiation and job status tracking
- File parsing, chunking, validation, normalization, enrichment
- Embedding generation and image processing pipeline
- Persistence to marketplace vehicle data and image records
- Queue/SQS/local orchestrator behavior and worker orchestration
- Concurrency, retries, idempotency, and stage graph execution
- Integration and e2e checks for ingest upload and job status

## 4.4 admin-service

- Admin reads and mutations
- Dashboard mapping and query DTO validation
- Internal clients for auth and notification service calls

- JWT strategy and roles guard
- Admin e2e flow for protected administration endpoints

## 4.5 notification-service

- Notification event handling
- Email template generation
- SES adapter behavior
- Notification retry sweeper logic
- Internal-service guard and repository behavior
- Notification e2e flow

## 4.6 api-gateway

- Public OpenAPI contract checks
- Internal OpenAPI contract checks
- CORS alignment
- nginx route/proxy validation
- Gateway health check
- Terraform scaffold consistency

the local NGINX gateway is tested for contract parity, but production routing is verified via Terraform checks against AWS API Gateway

## 4.7 web-frontend

- Auth storage and RequireAuth behavior
- Listings API contract tests
- Ingestion API contract tests
- Listing review and listing image flows
- Search hooks, search UI, pagination, vehicle formatting
- Dealer UI components, normalization badges, rejection reports
- Recommendation section, saved-vehicles hook, utility formatting tests

# 5. Deliverables

This section lists the artifacts produced by the AutoVaultLK test effort that provide direct, tangible benefit to project stakeholders and by which the success of testing will be measured.

- Unit test source files
- Integration test results
- Contract test results (OpenAPI/CORS/frontend)

- End-to-end (E2E) test results
- Jest test report
- Jest coverage report
- Test case document
- Test execution log
- Database verification screenshots
- API testing screenshots
- Image processing evidence
- Security testing results
- Performance results
- Load testing results
- Configuration/drift check results (nginx, CORS, OpenAPI, Terraform)
- Defect / failure log
- Final test summary

## 5.1 Test Evaluation Summaries

A test evaluation summary will be produced at the end of each testing cycle, capturing the number of test cases executed, passed, and failed for each testing category, along with the resulting pass percentage. Actual figures will be populated once the tests described in Section 3 have been executed completely. The structure to be used is shown below.

| **Category** | **Total** | **Passed** | **Failed** | **Pass %** |
|---|---|---|---|---|
| Unit Tests | TBD | TBD | TBD | TBD |
| Integration Tests | TBD | TBD | TBD | TBD |
| Contract Tests | TBD | TBD | TBD | TBD |
| E2E Tests | TBD | TBD | TBD | TBD |
| Security Tests | TBD | TBD | TBD | TBD |
| Performance Tests | TBD | TBD | TBD | TBD |
| Load Tests | TBD | TBD | TBD | TBD |
| Configuration Tests | TBD | TBD | TBD | TBD |
| Total | TBD | TBD | TBD | TBD |

## 5.2 Reporting on Test Coverage

Test coverage will be reported in two complementary forms: code coverage generated automatically by Jest, and functional coverage mapped against AutoVaultLK's key requirements

**Code coverage (from Jest):**

- Statements
- Branches
- Functions
- Lines

Each form of coverage is reported on a defined trigger rather than on request, so that coverage is a continuous signal rather than a milestone activity. The table below states when each is produced and who consumes it; the per-requirement table that follows carries the trigger for each requirement individually.

| **Coverage artefact** | **Produced by** | **Frequency / trigger** | **Consumer** |
|---|---|---|---|
| Code coverage (statements, branches, functions, lines) | Jest (backend services and gateway); Vitest with @vitest/coverage-v8 (web-frontend) | On every push and pull request, per service, in the CI workflow filtered to that service's paths. Reported against the service's threshold as a floor. | Service developers; QA lead |
| Functional coverage per requirement | QA lead, from the per-requirement table below | Refreshed on every merge to main, and reviewed in full at each phase transition (Section 11). | QA lead; project supervisor |
| Integration and contract suite results | Jest integration and contract configurations | On every push and pull request affecting the protected paths; integration suites additionally on any change to the migration set, grants or seed generators. | Service developers; database owner |
| Browser journey results (once Phase 3 is active) | The Playwright test environment | On every pull request into main, and nightly against the current staging build. | Frontend developer; QA lead |
| Performance baseline and load campaign results | k6 (thresholds as gates); CloudWatch during runs | Reduced smoke-load scenario on every pull request once introduced; full campaign on a scheduled basis against staging, and before any release. | QA lead; infrastructure owner |
| Security scan results | npm audit and dependency alerts in CI; OWASP ZAP against staging | Dependency scanning on every push and on new advisory publication; ZAP baseline scheduled, | Security reviewer; QA lead |

| | | | |
|---|---|---|---|
| | | authenticated active scan before each release. | |
| Configuration and drift check results | Terraform validate/plan; contract and scaffold suites | On every push affecting cloud-infrastructure, api-gateway or any service contract; and before every deployment to staging or production. | Infrastructure owner |

# 6. Risks, Dependencies, Assumptions, and Constraints

The following risks have been identified as potentially affecting the successful execution of this Test Plan,along with their mitigation and contingency strategies.

| **Risk** | **Impact** | **Mitigation Strategy** | **Contingency** |
|---|---|---|---|
| Insufficient test data | Medium | Create representative CSV / ZIP datasets | Generate additional data |
| Database unavailable | High | Verify DB availability before testing | Restart / restore DB |
| Service failure | High | Health checks on all services before test runs | Restart the failed service |
| Corrupt ZIP | High | Validate ZIP structure before processing | Mark the upload job as failed |
| Malicious ZIP | High | Path traversal protection on extraction | Reject the archive |
| Large inventory | Medium | Dedicated load testing ahead of release | Optimize worker processing |
| External AWS unavailable (S3/SQS/SES) | Medium | Mock / local ObjectStore, queue, and email adapter for testing | Retry or fall back to a local equivalent |
| Gateway/frontend contract drift | Medium | Automated OpenAPI contract tests in CI | Block release and resync contracts |

| | | | |
|---|---|---|---|
| Infrastructure (Terraform) drift | Medium | Regular Terraform plan/validate checks | Reconcile infrastructure state before release |
| Test environment differs from production | Medium | Document configuration precisely | Repeat key tests in the deployment environment |
| Insufficient test coverage | Medium | Regular Jest coverage analysis | Add missing tests before release |

# 7. References


- Jest Documentation https://jestjs.io/docs/getting-started (Accessed 2026)
- NestJS Testing Documentation https://docs.nestjs.com/fundamentals/testing (Accessed 2026)
- PostgreSQL Documentation https://www.postgresql.org/docs/ (Accessed 2026)
- pgvector Documentation https://github.com/pgvector/pgvector (Accessed 2026)
- TypeORM Documentation https://typeorm.io/ (Accessed 2026)
- Sharp (Image Processing) Documentation https://sharp.pixelplumbing.com/ (Accessed 2026)
- React Testing Library Documentation https://testing-library.com/docs/react-testing-library/intro/ (Accessed 2026)
- AWS Documentation (S3, SQS, SES) https://docs.aws.amazon.com/ (Accessed 2026)
- OpenAPI Specification https://spec.openapis.org/oas/latest.html (Accessed 2026)
- Terraform Documentation https://developer.hashicorp.com/terraform/docs (Accessed 2026)
- OWA SP Testing Guide https://owasp.org/www-project-web-security-testing-guide/(Accessed2026)
- AutoVaultLK Software Requirements Specification (SRS) project document
- AutoVaultLK Software Architecture Document (SAD) project document
- AutoVaultLK API Documentation project document


# 8. Responsibilities, Staffing and Training Needs

This section records who owns each testing technique, the staffing that technique assumes, and the tooling the team must become proficient in before it can be executed. It is referenced from Section 3 and is updated as each technique is adopted, so that the resourcing and environment implications of a technique are recorded at the point it enters the plan rather than discovered when it is first attempted.

# 8.1 Ownership by Technique

AutoVaultLK is developed by a small project team in which every member both implements and tests. Ownership below therefore denotes accountability for a technique's coverage, tooling and reported results not exclusive execution. Unit and component tests for a given change remain the responsibility of whoever authors that change, under the convention that a pull request carries the tests for the behaviour it introduces.

| **Technique** | **Owner role** | **Contributing roles** | **Principal responsibilities** |
|---|---|---|---|
| 3.3.1 Data and Database Integrity | Database owner | Service developers; QA lead | Maintain the migration, constraint and grant suites; own the seeded integration database and its fixtures; run and interpret EXPLAIN ANALYZE verification. |
| 3.3.2 Function | Service developers | QA lead | Author and maintain unit and API-level journey suites per service. |
| 3.3.3 User Interface | Frontend developer | QA lead (browser layer); UX reviewer | Maintain the Vitest component suites |
| 3.3.4 Performance Profiling | QA lead | Database owner; infrastructure owner | Define per-operation baselines from the non-functional requirements; profile query plans, vector search, LLM and embedding latency, and Lambda cold start; publish and version the baselines. |
| 3.3.5 Load | QA lead | Infrastructure owner | Author the k6 scenarios and thresholds; schedule and run load campaigns against staging; document the saturation point and the practical ingestion ceiling. |
| 3.3.6 Security and Access Control | Security reviewer | Service developers; infrastructure owner | Maintain the defined role permissions for buyer, dealer and administrator as the authoritative oracle; own the authorisation, abuse-protection and |

| | | | |
|---|---|---|---|
| | | | object-level suites; run and triage ZAP scans; review IAM and grant least-privilege. |
| 3.3.7 Failover and Recovery | Infrastructure owner | Database owner; service developers | Own fault-injection scenarios and the dependency-failure suites; rehearse backup restoration, point-in-time recovery and migration-bearing rollback; verify that alarms fire under their triggering faults. |
| 3.3.8 Configuration | Infrastructure owner | Gateway maintainer; frontend developer | Own Terraform validate/plan gating and the contract/drift suites across NGINX, CORS and OpenAPI; own the browser matrix definition. |

## 8.2 Cross-Cutting Responsibilities

- **QA lead** maintains this Master Test Plan; owns the defect log and the severity and priority definitions in Section 12; reports coverage on the triggers set out in Section 5.2; and decides suspension and resumption under Section 10.
- **Service developers** author unit, integration and API-level journey tests for the services they change; keep each service's CI gate green; author a reproduction test for every defect they fix.
- **Database owner** owns the migration set, the grants specification, seed and fixture generation, and the integration-database provisioning used in CI.
- **The infrastructure owner** owns the Terraform environments, the CI workflow definitions, the staging environment used for load and resilience work, and the monitoring and alarm definitions.
- **Security reviewer** owns the definition of role permissions across buyer, dealer and administrator, dependency and secret scanning, and the pre-release penetration pass
- **All team members** participate in exploratory testing sessions and in reviewing test evidence for the areas they did not author, so that no single person is the sole reviewer of their own work.

## 8.3 Training Needs

The techniques in Section 3.3 require tooling beyond what the team already uses routinely. The following are recorded as prerequisites, with the level at which each is needed, so that the learning is scheduled ahead of the phase that depends on it (Section 11).

| **Tool / skill area** | **Needed for** | **Current standing** | **Training approach** |
|---|---|---|---|
| Jest, ts-jest, @nestjs/testing; Vitest and React Testing Library | section 3.3.1–3.3.3, all levels | In routine use across all services | No additional training required; conventions are maintained by review. |

| | | | |
|---|---|---|---|
| TypeORM migrations, roles and grants; psql and EXPLAIN ANALYZE | section 3.3.1, integration level | Used by the database owner | Internal walkthrough so that at least two team members can provision, migrate, seed and inspect the integration database independently. |
| Playwright, Page Object Model, and Network Interception | section 3.3.3 and section 3.3.8, browser level | Not yet adopted | Focus on Playwright's trace viewer, auto-waiting, and API mocking capabilities instead of explicit-wait strategy and Grid configuration |
| k6 scripting and thresholds | section 3.3.4 and section 3.3.5 | Not yet adopted | Self-study plus a trial scenario against staging to calibrate virtual-user counts and threshold values before the first formal campaign. |
| OWASP Top 10 and ZAP; authenticated scan configuration | section 3.3.6 | Conceptual familiarity only | Work through the OWASP Testing Guide sections relevant to authentication, access control and injection; run ZAP in baseline mode against staging before attempting authenticated active scanning. |
| AWS operational tooling: CloudWatch, X-Ray, Lambda Insights, Step Functions execution history, RDS snapshot and point-in-time restore | section 3.3.4 and section 3.3.7 | Partial, held by the infrastructure owner | Pair the infrastructure owner with a second team member through one full recovery rehearsal, so restoration is not dependent on a single individual. |
| Terraform validate, plan and policy scanning (tfsec / Checkov) | section 3.3.8 | Terraform in use; policy scanning not yet adopted | Introduce policy scanning in report-only mode first, then promote to a gate once the existing findings are triaged. |
| axe-core and WCAG 2.1 AA; screen-reader operation | section 3.3.3 | Not yet adopted | Integrate axe-core into the component suite first for immediate feedback, then train on manual keyboard and screen-reader walkthroughs of the priority journeys. |

## 8.4 Staffing Assumptions and Constraints

- The team is small enough that roles overlap; each technique nonetheless has a single named owner so that accountability is unambiguous.
- Load testing, dynamic security scanning and recovery rehearsals each require exclusive use of the staging environment and are therefore scheduled rather than run on demand, as recorded in Section 11 and in the Special Considerations rows of Section 3.3.
- Recovery rehearsals additionally require credentials that permit restoring a database and corrupting state deliberately; these are held by the infrastructure and database owners only.

# 9. Test Environment Specification

This section consolidates the environment requirements that the Special Considerations rows in Section 3.3 refer to individually. Four environments are used. They differ in fidelity and in what may safely be done to them, and each technique is assigned to the least-privileged environment capable of producing a valid result.

| **Environment** | **Purpose and techniques hosted** | **Composition** | **Constraints** |
|---|---|---|---|
| Developer local | Unit and component suites (all levels' fast feedback); integration suites against a live database; contract suites. Dependency-failure injection at the container level for section 3.3.7. | Node.js 22. Docker Compose brings up PostgreSQL with the pgvector and text-search extensions on a mapped port, with the migration set, grants and deterministic seed data applied.LocalStack providing high-fidelity emulation of S3, SQS, and SES ; mail transport stubbed. | Not representative for performance work: instance sizing, network topology and managed-service limits differ from AWS. Integration suites skip rather than fail when no database is reachable, so an unprovisioned laptop never produces a misleading red build. |
| Continuous integration | Gating every change: typecheck, lint, unit, contract and API-level end-to-end suites, service build and container image build. Integration suites where a database service container is provisioned. The reduced smoke-load gate - once introduced. | GitHub Actions on Ubuntu runners, Node.js 22 with dependency caching, per-service workflows filtered to the paths they protect. PostgreSQL service container for integration suites. Docker for image builds. OIDC-based AWS credentials for the deployment workflow. | Ephemeral and non-interactive; no manual intervention or exploratory work. Not sized for load testing. Runtime budget is a real constraint, which is part of why the browser suite is kept to journeys only. |
| Staging (AWS) | Performance profiling (section 3.3.4) and all load, stress, spike, soak and volume testing (section 3.3.5). Authenticated dynamic security scanning | Terraform-provisioned mirror of the production topology at reduced scale: RDS PostgreSQL with pgvector, S3 buckets, SQS queues, SES in sandbox or a | Requires exclusive booking for load, scanning and recovery exercises, which must not overlap one another concurrent runs invalidate each other's |

| | | | |
|---|---|---|---|
| | (section 3.3.6). Fault injection, backup restoration, point-in-time recovery and rollback rehearsal (section 3.3.7). Cross-environment and browser-matrix configuration testing (section 3.3.8). | capture inbox, Lambda and Step Functions for the ingestion pipeline, API Gateway and the gateway/NGINX layer, CloudWatch metrics and alarms. Seeded to representative production inventory volume. | measurements. Destructive testing is permitted here and only here. |
| Production (AWS) | Post-deployment smoke verification and continuous monitoring only. | Terraform-provisioned from the same modules as staging, at production sizing. | No destructive testing, no load generation, no active security scanning, under any circumstances. Verification is limited to health checks and read-only smoke requests. Any defect found here is reproduced in staging before a fix is validated. |

## 9.1 Environment Parity and Known Divergence

Because LocalStack is now used, local/CI runs do provide early confidence in SQS delivery semantics and S3 behavior. Reserve the warning only for AWS IAM permissions and Lambda cold-starts, which still require Staging. Local and CI environments substitute local implementations for S3 and SQS and stub the mail transport. This divergence is deliberate it keeps the fast suites hermetic and offline-capable but it means that no local or CI run constitutes evidence about IAM permissions, S3 consistency behaviour, SQS delivery semantics, SES deliverability, or Lambda cold-start and memory behaviour. Those properties are verified only against staging, and the corresponding risk is recorded in Section 6 (test environment differs from production) with repetition of key tests in the deployment environment as its contingency.

# 10. Entry, Exit and Suspension Criteria

Criteria are stated per test level, since the levels are entered at different times and gate different things. Release-level criteria are stated separately at the end.

## 10.1 Entry Criteria

| **Level** | **Testing at this level may begin when** |
|---|---|
| Unit | The code under test compiles and passes typecheck and lint. The unit of behaviour has a stated specification an SRS clause, an FR entry, or an agreed acceptance criterion that can serve as an oracle independently of the implementation. |
| Integration | The relevant unit suites pass. A PostgreSQL instance with the required extensions is reachable, the full migration set applies cleanly, grants are |

| | |
|---|---|
| | applied, and deterministic seed data is loaded. The service roles used by the suites exist with their intended privileges. |
| Contract | The OpenAPI documents, NGINX configuration and CORS configuration under test are present and parseable. The consuming client or Terraform module that the contract is asserted against is available in the same revision. |
| End-to-end (API) | Unit, integration and contract suites for the participating services pass. The service boots against a seeded test database with its external dependencies stubbed or emulated. Authentication fixtures permitting each role under test are available. |
| End-to-end (browser) | The Playwright test env. and the fixture/seed API are available; a deployed staging build is reachable at a known version; the page objects for the journey under test exist; and the API-level journey for the same flow already passes so a browser failure implicates the interface rather than the backend. |
| Performance and load | Functional correctness at the API level is established, since measuring a broken path is meaningless. Staging is provisioned at representative data volume, booked exclusively for the window, and baselines (for load) or non-functional targets (for profiling) are documented in advance. |
| Security | The role permissions for buyer, dealer and administrator are current and agreed. For scanning, staging is reachable, authenticated scan credentials exist, and the scan window does not overlap a load or recovery exercise. |
| Failover and recovery | The system is functionally correct under normal conditions. A restorable backup or snapshot exists. The exercise is scheduled, and the infrastructure and database owners are available for the window. |

## 10.2 Exit Criteria

| **Level** | **Testing at this level is complete when** |
|---|---|
| Unit | Every unit in scope has a suite; each has at least one negative or error-path case; boundary values are covered at and either side of each limit; the coverage threshold for the service is met with any exclusion justified; and the suite passes repeatedly and in any order, not once. |
| Integration | Every cross-schema read, every upsert path and every pipeline stage transition in scope is covered |

| | |
|---|---|
| | by a live-database test; migrations are proven forward, idempotent and reversible; every constraint and unique index has a negative test; and every profiled query is confirmed to use its intended index. |
| Contract | Both OpenAPI documents validate; every gateway route resolves to a documented operation and a live service path; CORS origins match across the gateway and every service; the NGINX configuration passes its own parser; every frontend client module has a contract test; and the normalisation/embedding parity check passes. |
| End-to-end (API) | Every functional requirement is exercised by at least one passing journey , every protected endpoint has both an authorised and an unauthorised journey; and side effects are asserted in the database rather than inferred from the response. |
| End-to-end (browser) | Every priority journey passes across the supported browser matrix; the suite is stable enough to gate merges, with flakiness tracked and quarantined rather than masked by blanket retry; and every failure produces a screenshot, DOM snapshot and console log. |
| Performance and load | A documented baseline exists for every critical operation; percentile targets are met under expected load; the saturation point is documented and degradation past it is predictable; no memory growth or connection-pool leak is observed over the soak period; and the practical ingestion batch ceiling is documented. |
| Security | Every protected endpoint is covered for every role including denial cases ; object-level authorisation is proven by identifier substitution on every owned resource; token rotation, replay and expiry behaviour is proven; and every high or critical finding is remediated or formally risk-accepted with recorded rationale. |
| Failover and recovery | Every external dependency has a failure test asserting a clear error, a diagnostic log entry and no partial write; replay produces exactly-once effect; an interrupted ingestion job always resumes or fails cleanly with accurate status; a timed restore meets the stated RPO and RTO; and every critical alarm is proven to fire under its triggering fault. |

Across every level, exit additionally requires that all defects at critical and high severity (Section 12) are resolved and verified by a retained reproduction test, and that medium and low findings are either resolved or formally accepted with recorded rationale.

## 10.3 Release Exit Criteria

- Every functional requirement traced to at least one passing test
- The full automated suite green in CI across all services, with no required check bypassed.
- Priority browser journeys passing across the supported browser matrix.
- Performance targets met, with baselines documented and the saturation point known.
- No unremediated critical or high security finding; all others formally risk-accepted.
- Disaster recovery rehearsed within the stated RPO and RTO, and a migration-bearing rollback proven.
- No outstanding serious or critical accessibility violation on any page.
- Post-deployment smoke verification passing in staging and in production.

## 10.4 Suspension Criteria and Resumption Requirements

Testing at a level is suspended when continuing would produce results that cannot be trusted or acted upon. Suspension is a decision of the QA lead, recorded in the defect log with the triggering condition, and it applies to the affected level only other levels continue unless they share the blocking cause.

| **Suspension trigger** | **Resumption requires** |
|---|---|
| The build is broken, or typecheck or lint fails on the branch under test. | A green build on the branch; the failing gate passing again. |
| A blocking defect prevents meaningful further execution at the level for example an authentication failure that makes every downstream journey unreachable. | The blocking defect was fixed and verified, and the suites that were unreachable re-executed from the start rather than resumed mid-run. |
| The required environment is unavailable or unrepresentative: no reachable database for integration suites, staging unprovisioned or mis-sized, Grid unavailable for browser suites. | The environment provisioned and verified against Section 9, including migrations, grants and seed data where applicable. |
| Requirement instability leaves the level without a usable oracle; the specification under test is being actively rewritten. | The affected requirements are stable and re-baselined, and the affected tests and traceability entries updated to match. |
| Defect arrival rate at a level is high enough that further execution is producing duplicates of one underlying cause rather than new information. | The underlying cause was diagnosed and fixed, then the level re-entered under its Section 10.1 entry criteria. |
| A staging exercise (load, scan or recovery) is found to overlap another, invalidating measurements. | Exclusive rebooking of the environment and a full re-run; partial results from the overlapping window are discarded, not reported. |
| Test data is discovered to be non-deterministic, stale, or contaminated with non-synthetic content (Section 13). | Fixtures regenerated deterministically and re-committed; affected suites re-executed against the corrected data. |

# 11. Test Schedule and Phasing

Testing is phased so that each technique is introduced when its prerequisites functional stability, environment availability, and the training recorded in Section 8.3 are in place. Phases overlap: once a technique is introduced it continues for the remainder of the project as part of the regression burden, so the table records the iteration in which each technique is established rather than a window in which it is finished. Phase 1 is in progress.

| Phase | Focus | Techniques established | Levels active | Environment | Exit gate |
|---|---|---|---|---|---|
| 1. Foundation (in progress) | Breadth of functional and data coverage; contract and static gates wired into CI. | section 3.3.1 Data and Database Integrity; section 3.3.2 Function; section 3.3.3 User Interface (component layer only); section 3.3.8 Configuration (contract and drift suites). | Unit, Integration, Contract, End-to-end (API) | Local; CI | Section 10.2 exit for the unit, integration and contract levels; every service gated in CI. |
| 2. Depth and hardening | Systematic negative and boundary coverage; authorisation proven exhaustively. | section 3.3.6 Security and Access Control (application and system level, excluding dynamic scanning); constraint-negative and migration round-trip suites under section 3.3.1. | Unit, Integration, Contract, End-to-end (API) | Local; CI | Full role matrix covered including denial cases; object-level authorisation proven on every owned resource; dependency and secret scanning active in CI. |
| 3. Browser automation | Real-browser journeys and the supported browser matrix. | section 3.3.3 User Interface (browser layer); section 3.3.8 Configuration (browser matrix). | End-to-end (browser) | Staging; CI (Grid) | Every priority journey passing across the matrix; suite stable enough to gate merges; failure artefacts captured. |
| 4. Performance and resilience | Quantified baselines and proven recovery. | section 3.3.4 Performance Profiling; section 3.3.5 Load; section 3.3.7 Failover and Recovery. | End-to-end (API and browser), with unit-level fault injection | Staging (exclusively booked) | Baselines documented and met; saturation point known; timed restore within RPO and RTO; migration-bearing rollback proven; critical alarms proven to fire. |

| | | | | | |
|---|---|---|---|---|---|
| 5. Release assurance | Independent verification and sign-off. | section 3.3.6 (authenticated dynamic scanning and manual penetration pass); accessibility conformance audit | All levels, confirmatory | Staging; production (smoke only) | Section 10.3 release exit criteria met and signed off. |

## 11.1 Continuous Activities

Two activities are not phased because they run throughout: regression execution in CI on every change, and defect triage against the severity and priority definitions in Section 12. Each phase transition is also the point at which this plan is reviewed and revised, so that it continues to describe the framework actually in force.

# 12. Defect Severity and Priority Definitions

The defect log referenced in Section 5 classifies every finding on two independent scales. Severity describes the technical consequence of the defect if it reaches production, and is assigned by whoever reports it. Priority describes the urgency of fixing it relative to other work, and is assigned by the QA lead in consultation with the owner of the affected area. The two are deliberately separate: a cosmetic defect on the dealer registration page may be low severity and high priority because it is the first screen a paying user sees, while a critical-severity defect in an unreleased feature may be scheduled behind shipping work.

## 12.1 Severity Scale

| **Severity** | **Definition** | **AutoVaultLK examples** | **Handling** |
|---|---|---|---|
| S1 – Critical | Data loss or silent data corruption; a security or authorisation bypass; or complete unavailability of a core flow with no workaround. | An ingestion job reporting success over a partially written batch. Re-upload duplicating a dealer’s entire inventory. One dealer able to read or mutate another dealer’s listings or upload jobs. Authentication bypass, or a token accepted after revocation. Search or login unavailable platform-wide. | Blocks release unconditionally. Fixed immediately; a reproduction test is retained permanently. May trigger suspension of the affected level under Section 10.4. |
| S2 – High | A core flow fails or produces incorrect results, but a workaround exists or the blast radius is bounded. | Valid CSV rows rejected, or invalid rows accepted. Semantic search returning incorrectly ranked results, or the | Blocks release. Fixed within the current iteration; reproduction test retained. |

| | | | |
|---|---|---|---|
| | | deterministic fallback failing when the LLM path is down. Notification never delivered for a completed job. An admin action not written to the audit log. | |
| S3 – Medium | A non-core function is incorrect, or a core function is incorrect only in an edge case; the primary journey still completes. | Pagination miscounting on the final page of results. A rejections report mis-attributing a pipeline stage. An empty-state message absent where a list has no items. Incorrect currency or mileage formatting in one view. | Does not block release on its own. Scheduled by priority; must be resolved or formally accepted with recorded rationale before release under Section 10.2. |
| S4 – Low | Cosmetic or textual; no functional impact. | Inconsistent spacing or heading weight between dealer console pages. A typographical error in a validation message. A tooltip not matching the wording in the design documentation. | Logged and batched. Resolved opportunistically or formally accepted. |

## 12.2 Priority Scale

| **Priority** | **Definition** |
|---|---|
| P1 – Immediate | Work stops on other tasks until this is resolved. Applied to any S1, and to any defect blocking a test level under Section 10.4. |
| P2 – High | Resolved within the current iteration, ahead of new feature work in the affected area. |
| P3 – Normal | Scheduled into a subsequent iteration in the ordinary course of work. |
| P4 – Low | Addressed opportunistically, typically alongside other work in the same area, or formally accepted. |

## 12.3 Required Fields and Lifecycle

Every entry in the defect log records: identifier; title; severity and priority; the service or component affected; the test level and technique that found it; the environment; steps to reproduce; expected and actual behaviour; the requirement or oracle violated; and the identifier of the reproduction test once written. A defect moves through Open, In Progress, Resolved, Verified and Closed; it reaches Verified only when the reproduction test fails against the original revision and passes against the fix, and that test is then retained permanently as part of the regression suite.

# 13. Test Data Policy

All test data used at every level and in every environment is synthetic. No real dealer inventory, no real buyer account, and no personal data of any kind from a production or live-pilot source is copied into a test environment, used in a fixture, or committed to the repository. This is a firm constraint rather than a preference: dealer inventory is commercially sensitive, buyer records are personal data, and a test environment is a weaker security boundary than production by design.

## 13.1 Principles

- **Synthetic only.** Vehicle records, dealer profiles, buyer accounts, images and credentials are generated, never extracted. Where a realistic distribution matters, make and model frequency, price and mileage spread, image count per vehicle the generators are parameterised to approximate the real distribution without reproducing any real record.
- **Deterministic.** Generators are seeded so that the same inputs produce the same dataset on every run and on every machine. A test that passes locally and fails in CI must not be explicable by different data. Randomness without a fixed seed is treated as a defect in the fixture.
- **Version controlled.** Fixtures and generator scripts live in the repository alongside the suites that consume them, so a dataset change is reviewable and attributable in the same way as a code change, and any revision can be reproduced exactly.
- **Owned by the suite.** Each suite establishes the data it needs and cleans up or scopes it afterwards, so suites remain order-independent and repeatable. No suite depends on data left behind by another.
- **No real credentials.** Secrets, tokens and API keys in test data are dummy values. Credentials for external providers used in staging are held in the secret store, never in fixtures, source or logs.

## 13.2 Data by Environment

| Environment | Data provision | Volume |
|---|---|---|
| Developer local | Docker Compose PostgreSQL, migrated and seeded by the deterministic vehicle, image and embedding generators. Uploads exercised with generated CSV and ZIP fixtures, including deliberately malformed variants. | Small sized for fast feedback, not for realism. |
| Continuous integration | Provisioned fresh per run from the same migrations, grants and generators; discarded afterwards. | Small and fixed, so runtime is predictable. |
| Staging (AWS) | Generated to approximate anticipated production inventory scale, so that performance and load results are representative rather than optimistic. Refreshed by regeneration, never by copying from production. SES is directed at a capture inbox or sandbox rather than real recipients. | Representative of anticipated production volume. |
| Production (AWS) | Real data. No test data is introduced, and no test account is | Not applicable read-only smoke verification only. |

| | | |
|---|---|---|
| | created. | |

## 13.3 Adversarial and Negative Data

Several techniques in Section 3.3 require data that is deliberately invalid or hostile: malformed and oversized CSV rows, duplicate VINs, out-of-range year, mileage and price values, invalid enum values, orphaned foreign keys, corrupt ZIP archives, archives containing path-traversal entries, payloads carrying SQL and script metacharacters, and text crafted to attempt prompt injection through the normalisation stage. These fixtures are generated and committed alongside the valid ones, clearly labelled as adversarial, and are confined to test environments. They are never introduced into production, and the archive-abuse fixtures in particular are handled only by the suites that assert extraction resists them.